

2 System Structures

An Example

- **Ask OS to load the editor**
  - User Interface
- **Load editor into memory**
  - Find on disk drive
    - File System
    - I/O Devices
  - Bring into Memory
- **Run the program**
  - User enters data
    - Keyboard & mouse input
  - Saves the file

Operating System Services

- **Operating systems provide an environment for execution of programs and services to programs and users**
- **One set of operating-system services provides functions that are helpful to the user:**
  - **User interface -** Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
  - **Program execution -** The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations -** A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation -** The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

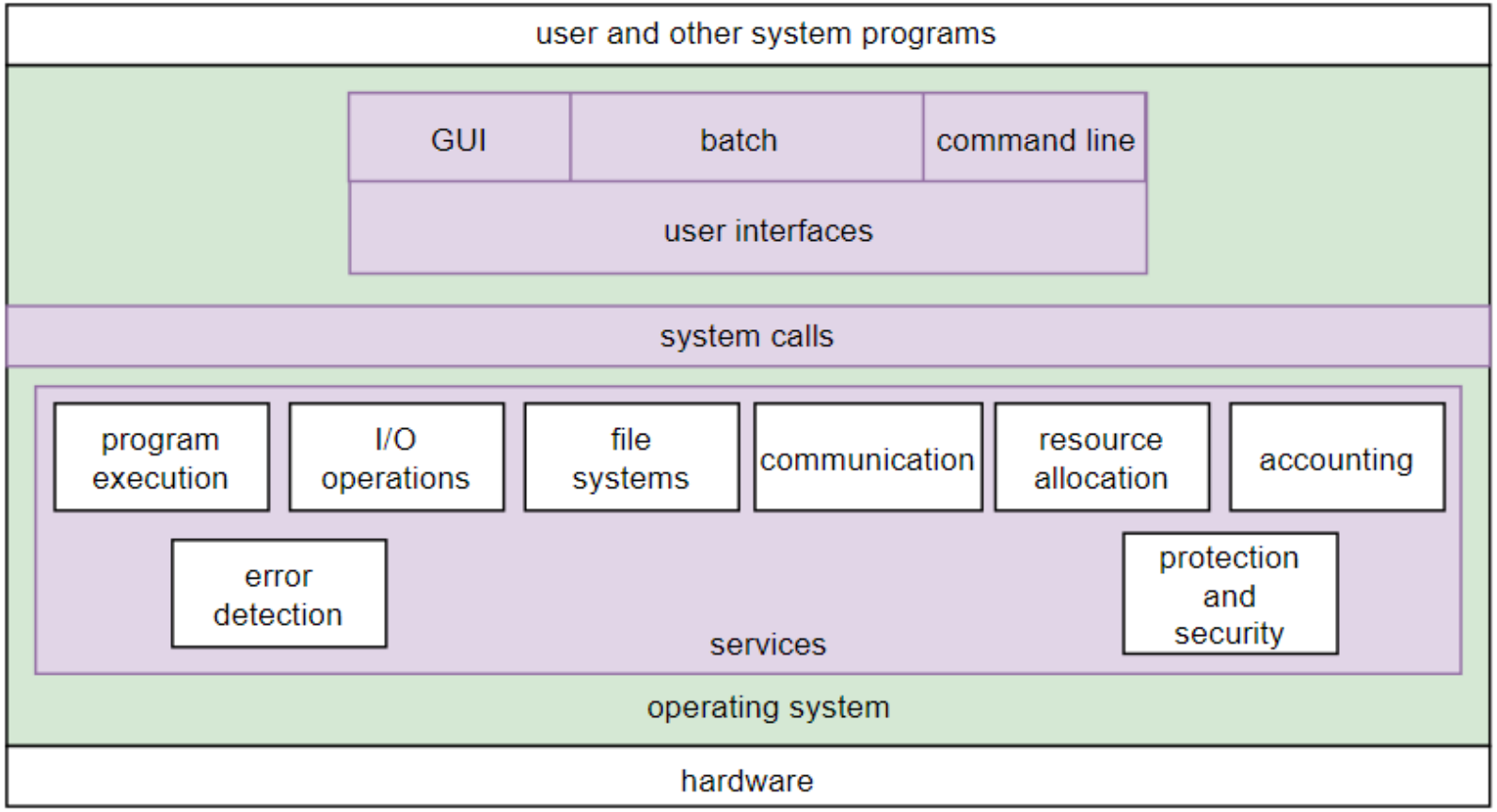
Operating System Services (Cont.)

- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services (Cont.)

- **Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing**
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
    - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

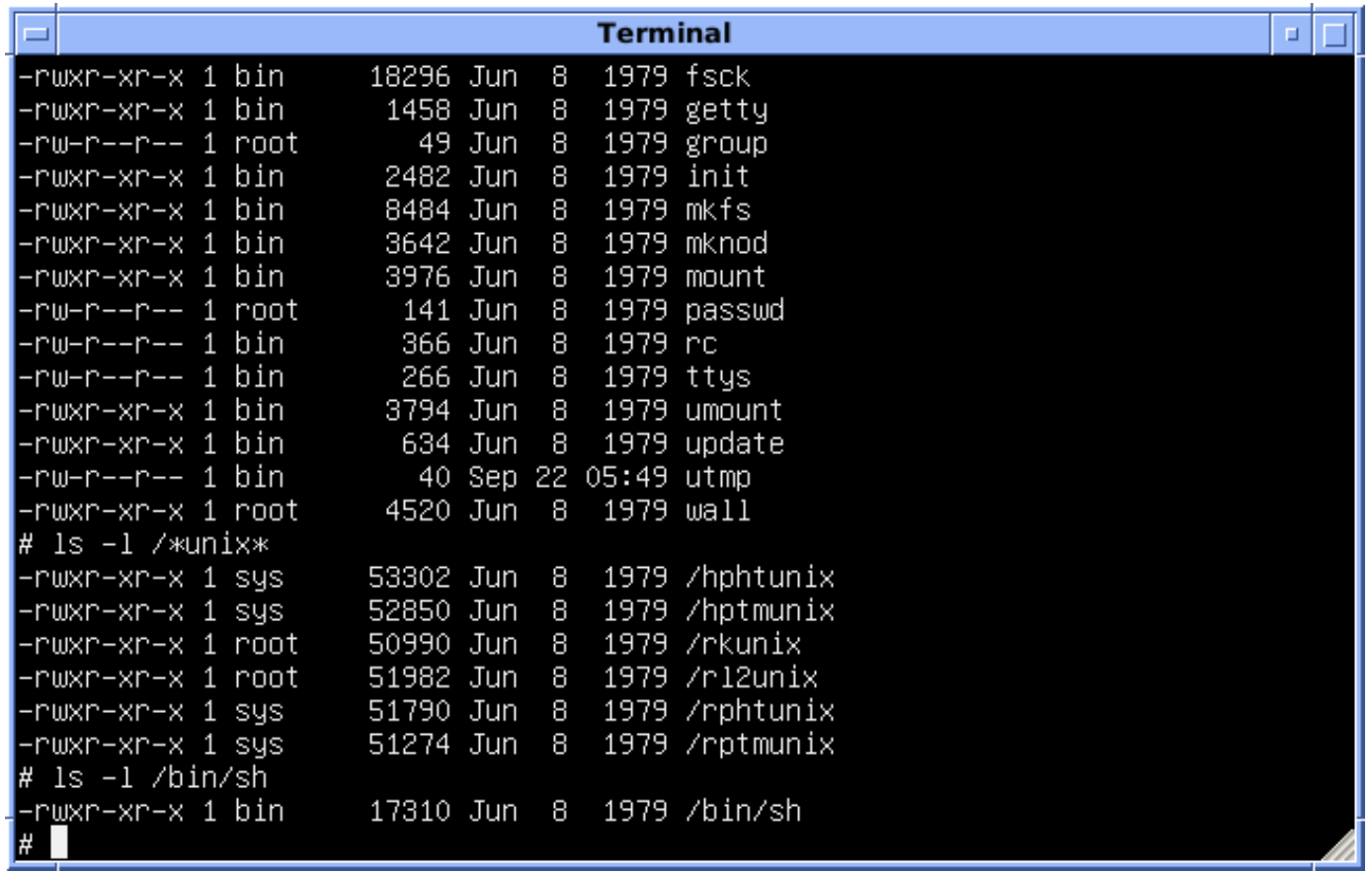
A View of Operating System Services



User Operating System Interface - CLI

- **CLI or command interpreter allows direct command entry**
  - Sometimes implemented in kernel, sometimes by systems program
  - Sometimes multiple flavors implemented – **shells**
  - Primarily fetches a command from user and executes it
    - Sometimes commands built-in, sometimes just names of programs
      - If the latter, adding new features doesn't require shell modification

Bourne Shell Command Interpreter



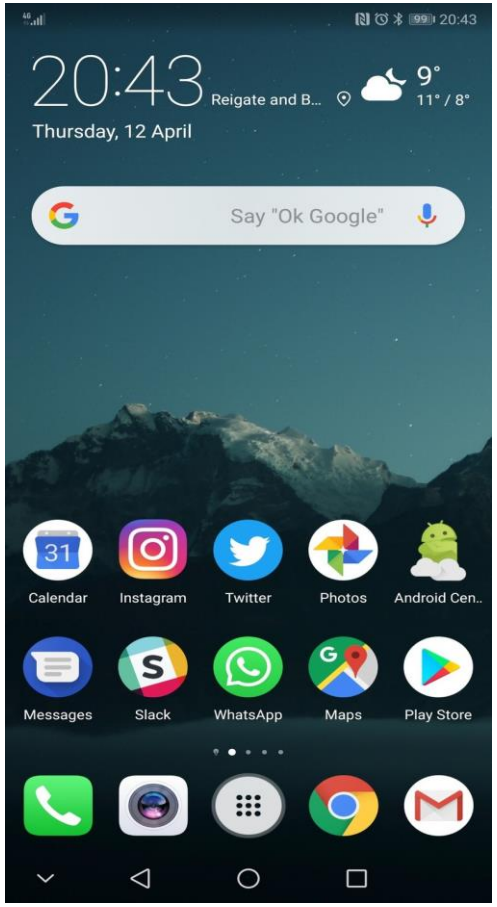
User Operating System Interface - GUI

- **User-friendly desktop metaphor interface**
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- **Many systems now include both CLI and GUI interfaces**
  - Microsoft Windows is GUI with CLI “command” shell
  - Apple macOS is “Aqua” GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

Touchscreen Interfaces

Touchscreen devices require new interfaces

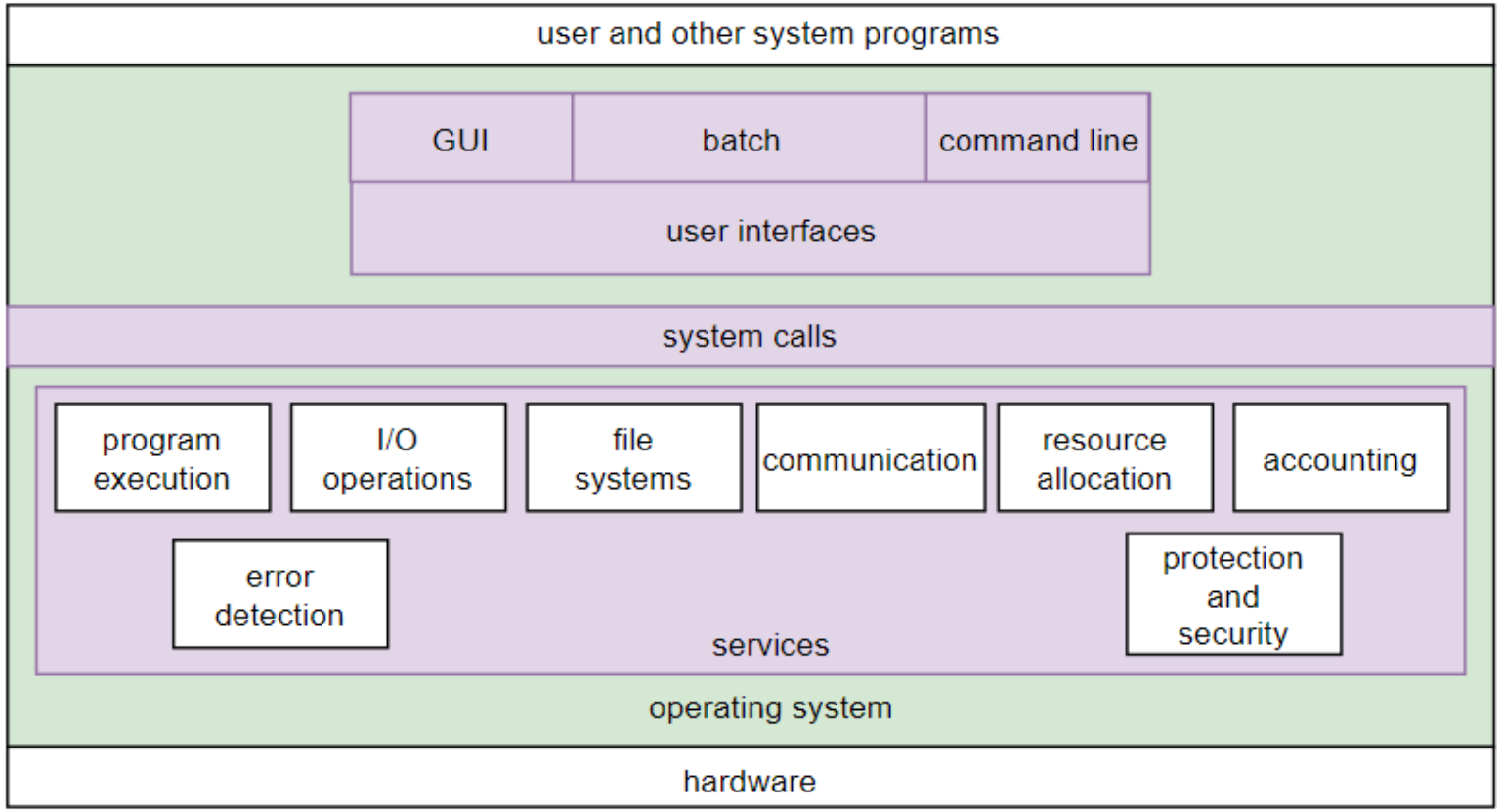
- Mouse not possible or not desired
- Actions and selection based on gestures
- Virtual keyboard for text entry



The macOS GUI



A View of Operating System Services

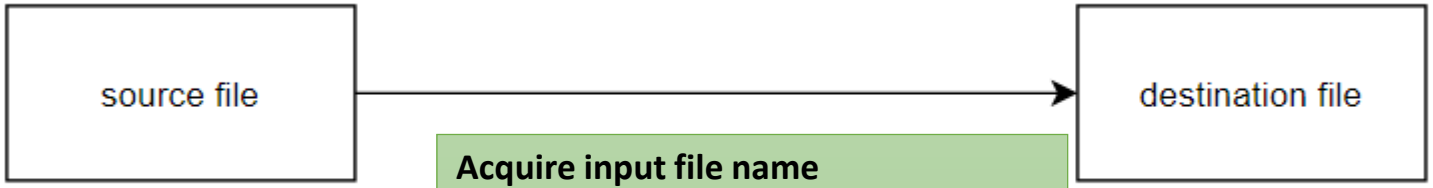


System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and macOS), and Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

Example of System Calls

System call sequence to copy the contents of one file to another file



Acquire input file name
Write prompt to screen
Accept input
Acquire output file name
Write prompt to screen
Accept input
Open the input file
If file doesn't exist, abort
Create output file
If file exists, abort
Loop
Read from input file
Write to output file
Until read fails
Close output file
Write completion message to screen
Terminate normally

Example of Standard API

Source：https://linux.die.net/man/2/read

As an example of a standard AIP, consider the read() function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

man read

on the command line. A description of this API appears below:

Name

read - read from a file descriptor

Synopsis

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

Description

read() attempts to read up to *count* bytes from file descriptor *fd* into the buffer starting at *buf*.

...

Return Value

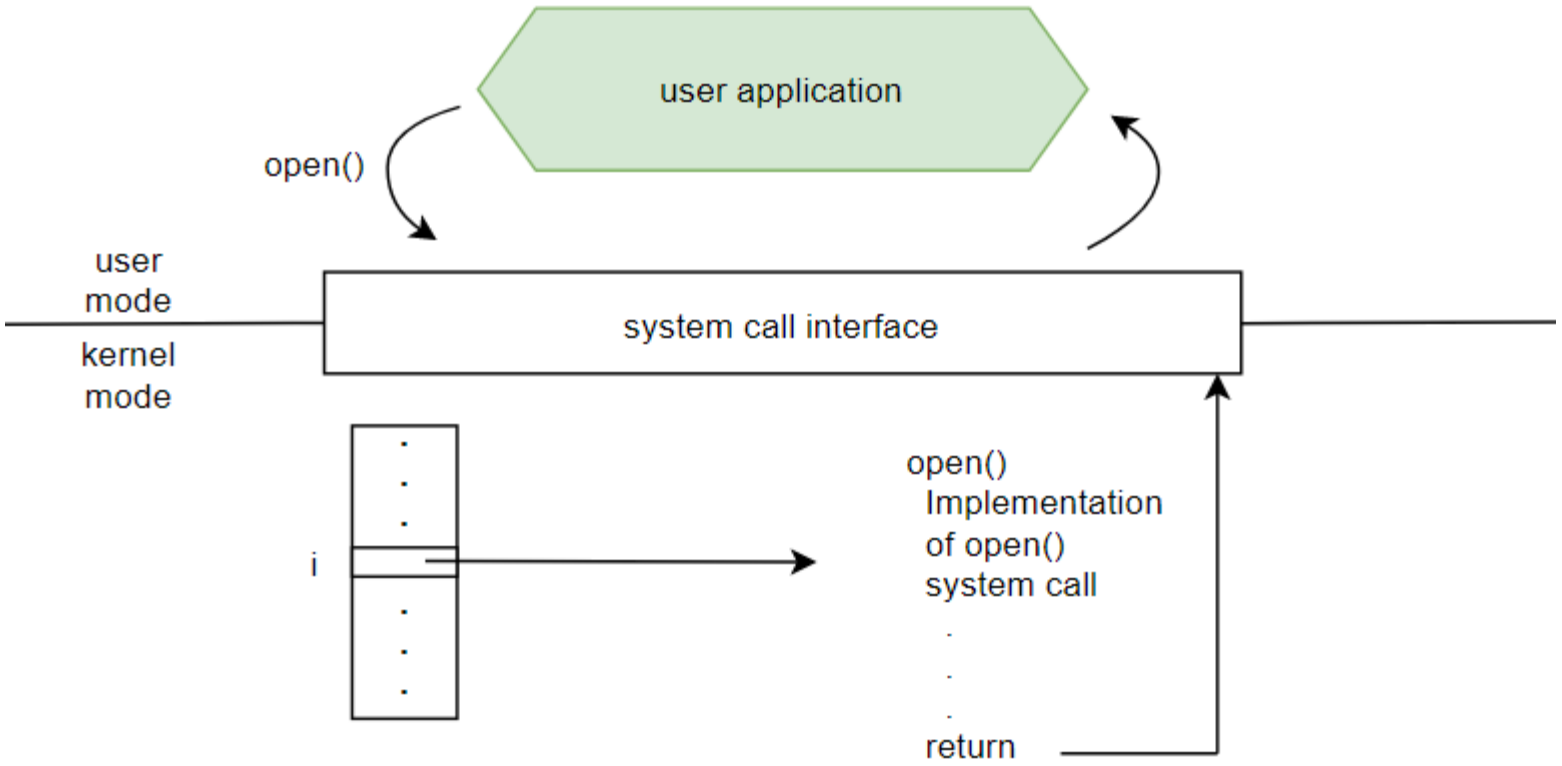
On success, the number of bytes read is returned (zero indicates end of file), and the file position is advanced by this number. It is not an error if this number is smaller than the number of bytes requested;

...

System Call Implementation

- **Typically, a number associated with each system call**
  - **System-call interface** maintains a table indexed according to these numbers
- **The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values**
- **The caller need know nothing about how the system call is implemented**
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship

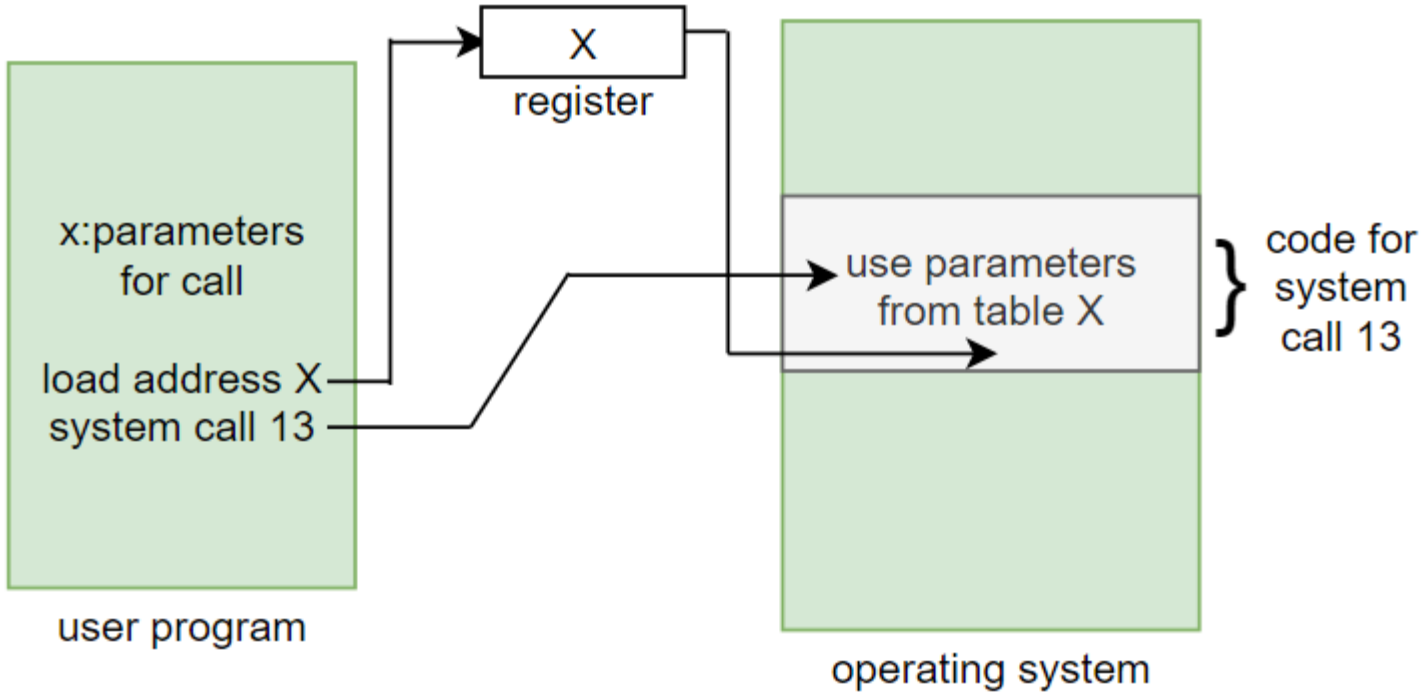


System Call Parameter Passing

- **Often, more information is required than simply identity of desired system call**
  - Exact type and amount of information vary according to OS and call

- **Three general methods used to pass parameters to the OS**
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls

- **Process control**
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes

Types of System Calls

- **File management**
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- **Device management**
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System Calls (Cont.)

- **Information maintenance**
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- **Communications**
  - create, delete communication connection
  - send, receive messages, if **message passing model**, to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System Calls (Cont.)

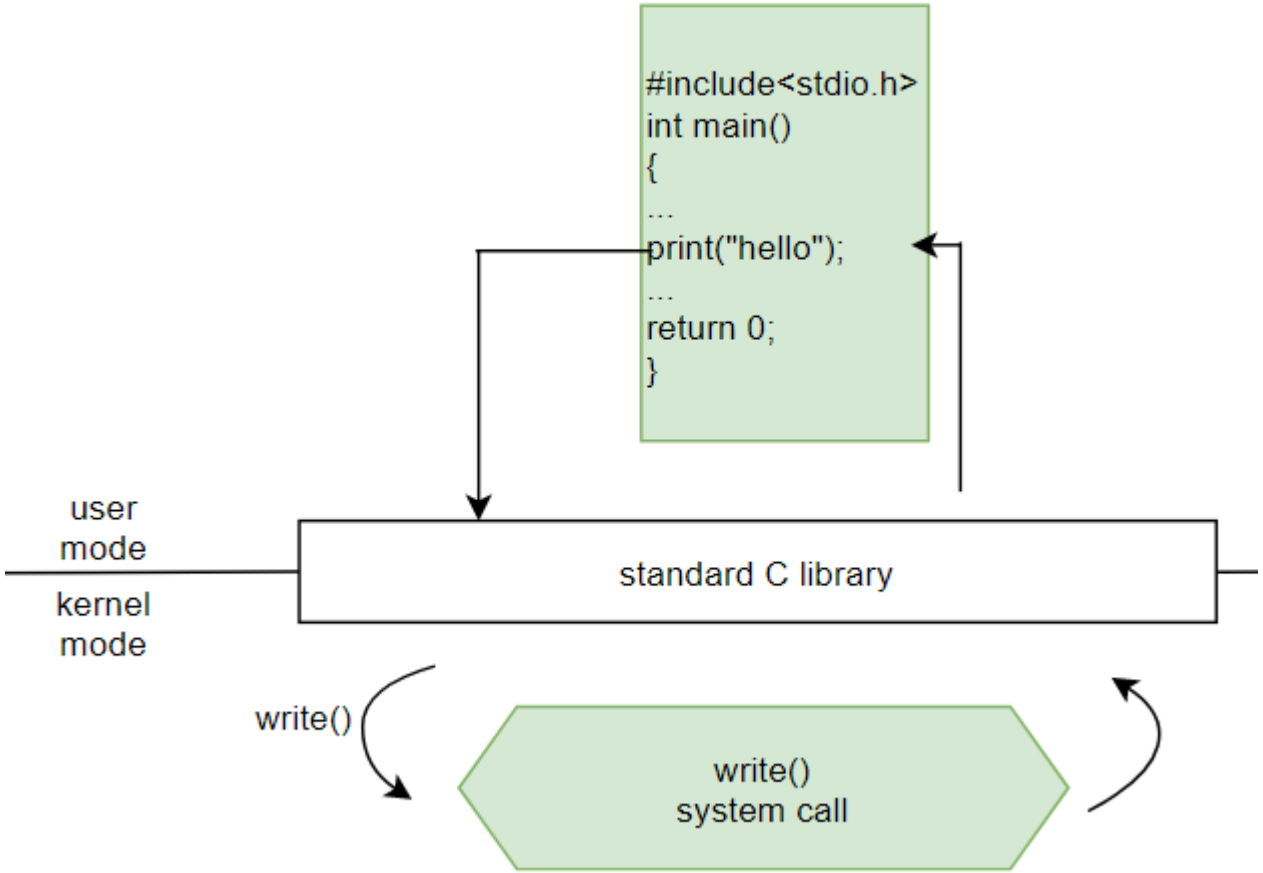
- **Protection**
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()

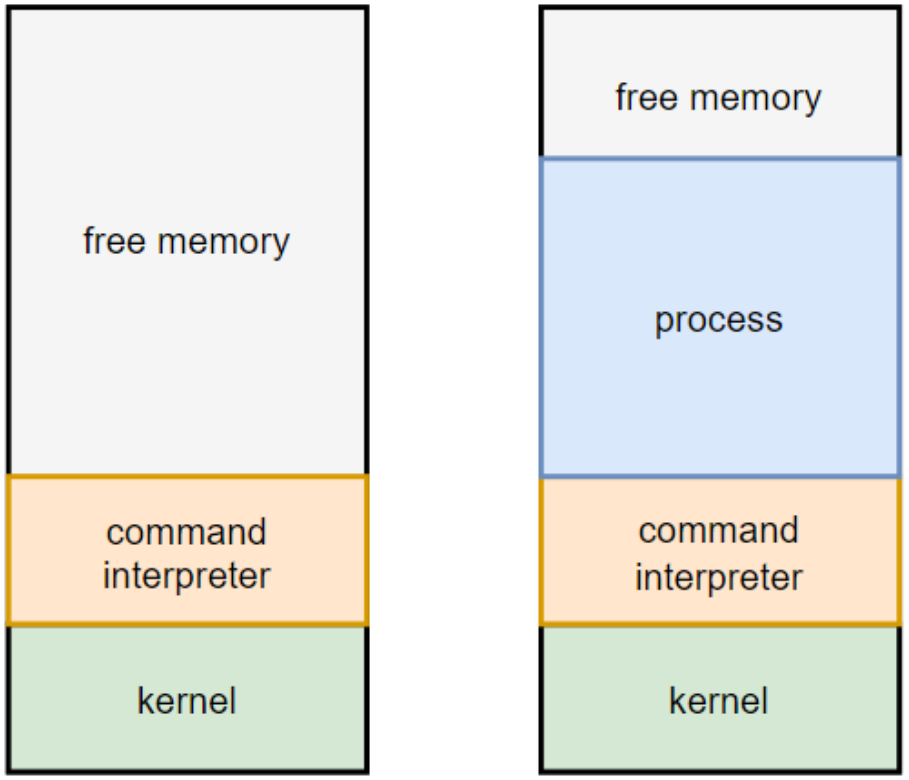
Standard C Library Example

- **C program invoking printf() library call, which calls writ**

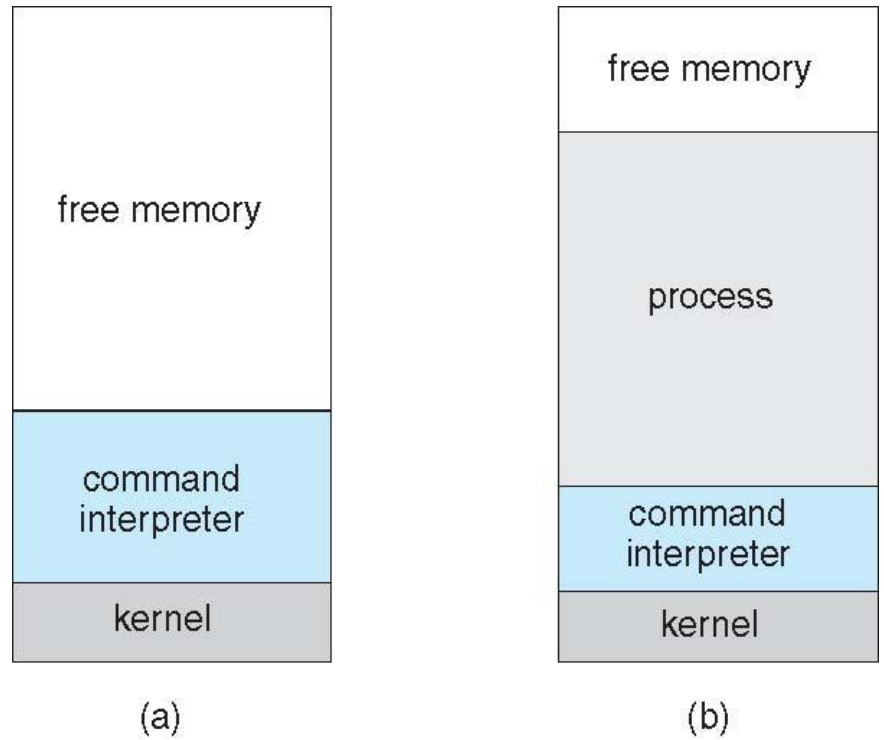


Example: MS-DOS

- **Single-tasking**
- **Shell invoked when system booted**
- **Simple method to run program**
  - No process created
- **Single memory space**
- **Loads program into memory**
- **Program exit → shell reloaded**

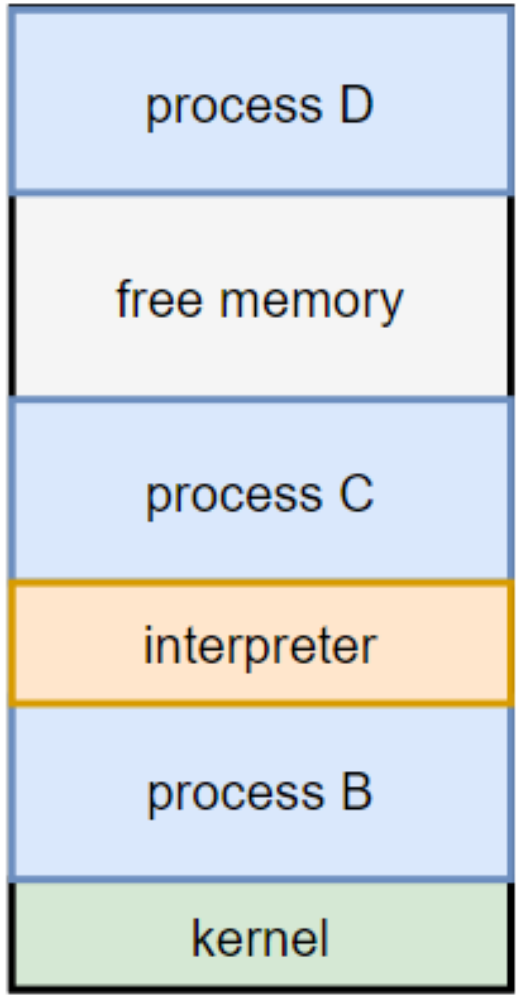


(a) At system startup (b) running a program


free memory
command interpreter
kernel
(a)
free memory
process
command interpreter
kernel
(b)

Example: FreeBSD

- **Unix variant**
- **Multitasking**
- **User login → invoke user's choice of shell**
- **Shell executes fork() system call to create process**
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- **Process exits with code**
  - 0 for no error, or
  - > 0 for error code



System Programs

System programs provide a convenient environment for program development and execution.

They can be divided into:

- File manipulation
- Status information sometimes stored in a File modification
- Programming language support
- Program loading and execution
- Communications
- Background services
- Application programs

Most users' view of the operation system is defined by system programs, not the actual system calls

System Programs

- **Provide a convenient environment for program development and execution**
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management**
  - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve

System Programs (Cont.)

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support**
  - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**
  - Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications**
  - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

System Programs (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don’t pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke

Operating System Design and Implementation

- **Design and Implementation of OS not "solvable", but some approaches have proven successful**
- **Internal structure of different operating systems can vary widely**
- **Start by defining goals and specifications**
- **Affected by choice of hardware, type of system**
- **User goals and system goals**
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

OS Goals

- **Older Goal to optimize hardware use**
  - Machine $1M
  - Users $nK per year
- **PC Goal to optimize user**
  - Machine $n hundreds
  - Users $mK per year
  - Work sometimes irreplaceable
    - Artists, writers
    - Engineers

Operating System Design and Implementation (Cont.)

- **Important principle to separate**

  Policy: *What* will be done?

  Mechanism: *How* to do it?

- **Mechanisms determine how to do something, policies decide what will be done**
  - The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later
- **Specifying and designing OS is highly creative task of software engineering**

Backward Compatibility

- **The price of success**
- **Constrains development**
- **Even bugs sometimes live on**
  - “It’s not a bug—it’s a feature!”

Implementation

- **Much variation**
  - Early OSs in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- **Actually usually a mix of languages**
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- **More high-level language easier to port to other hardware**
  - But slower
- **Emulation can allow an OS to run on non-native hardware**

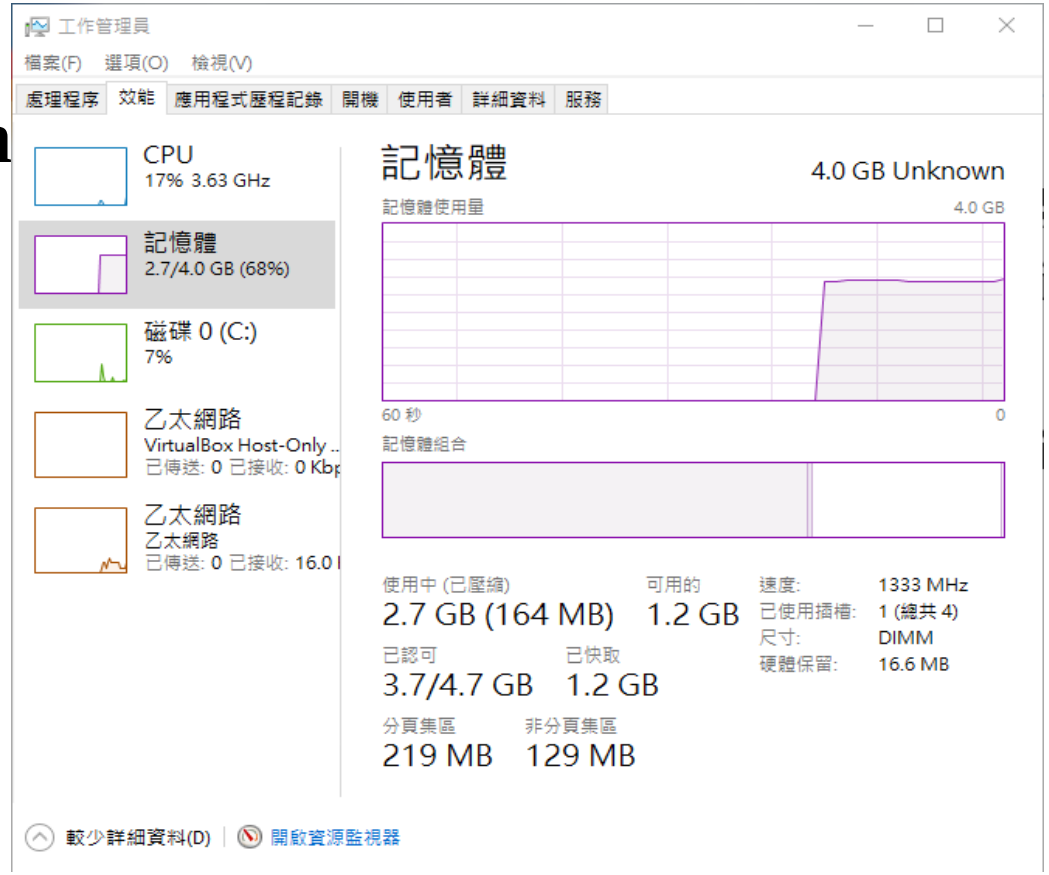
Operating-System Debugging

- **Debugging is finding and fixing errors, or bugs**
- **OSs generate log files containing error information**
- **Failure of an application can generate core dump file capturing memory of the process**
- **Operating system failure can generate crash dump file containing kernel memory**
- **Beyond crashes, performance tuning can optimize system performance**
  - Sometimes using ***trace listings*** of activities, recorded for analysis
  - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, "`top`" program or Windows Task Manager



Operating System Generation

- **Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site**

- **`SYSGEN` program obtains information concerning the specific configuration of the hardware system**
  - Used to build system-specific compiled kernel or system-tuned
  - Can general more efficient code than one general kernel

System Boot

- **When power initialized on system, execution starts at a fixed memory location**
  - Firmware ROM used to hold initial boot code
- **Operating system must be made available to hardware so hardware can start it**
  - Small piece of code – **bootstrap loader**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
- **Common bootstrap loader, `GRUB`, allows selection of kernel from multiple disks, versions, kernel options**
- **Kernel loads and system is then running**

Operating System Structure

- **General-purpose OS is very large program**
- **Various ways to structure one as follows**

Early PCs

- **Early PCs had few resources**
- **Initially no OS**
  - maybe a ROM Monitor

A Monitor Call

- Load character into register E
- Load function # into register C
- Cause interrupt 5
- [monitor outputs character]
- Load status into register A
- Return to caller

The Software Crisis

- **Even with same CPU–**
  - Each monitor was unique
  - Editors & compilers were unique
- **Software requires big development**
- **Broad market required to justify**
- **Broad market not there**

Software Explosion

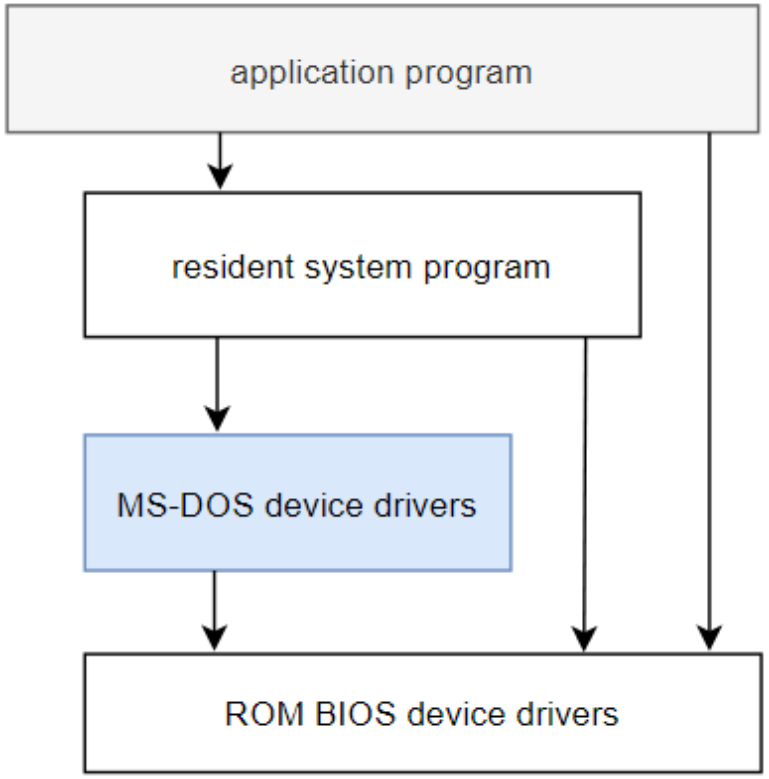
- **Thousands of programs in short time**
  - Tools
    - Editors, compilers, debuggers
  - Applications
    - Word processors
    - Flat file data systems
    - Accounting
  - Games

Application Programs

- **Basic IO needs**
  - Read keyboard
  - Write to video
  - Printed output
  - Disk file system
    - Create a file
    - Read & write file
    - Close file
- **OS hides device differences**

Simple Structure

- **I.e. MS-DOS – written to provide the most functionality in the least space**
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



System Characteristics

- **Main memory quite small**
  - 64 KB maximum – 4 to 16 KB typical
- **Standardized disk format**
- **Interrupts for I/O only**
  - single program so
    - no switching
    - no protection

Enhanced Keyboard Functions

- **Modified keys**
  - CTL/S to save a file
- **Not supported by BDOS**
- **Programs resorted to BIOS or direct hardware**
  - Just as easy to write
  - Programs no longer portable
  - Conflicts with other programs

Video Hardware

- More variants than keyboard
- BDOS support very limited
- BDOS support very slow (*100)
- Video refreshed from main RAM
- Direct access fast & easy
- Benefits & drawbacks as with keyboard

What *really* is the Operating System?

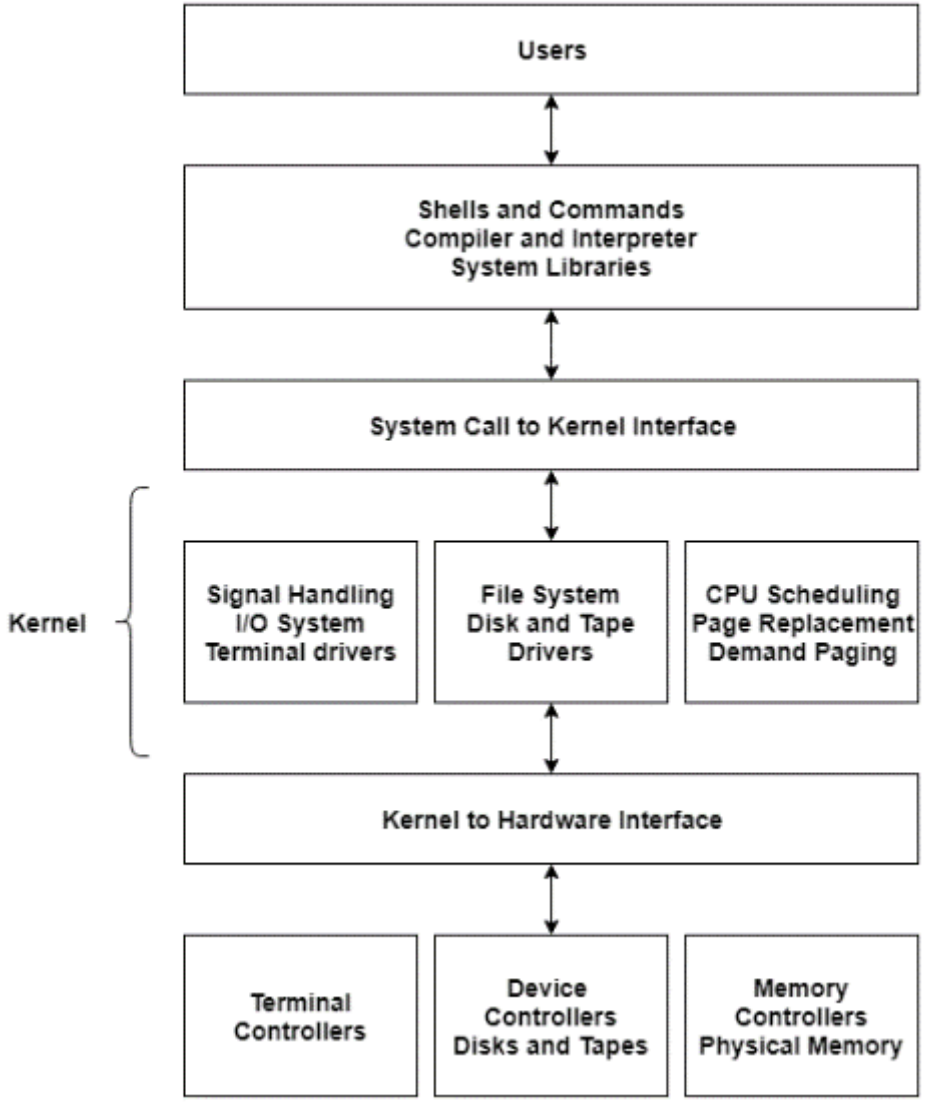
- **Many programs skipped BDOS**
  - video & keyboard
- **A portable file system**
- **75% of OS calls were for disk**

UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

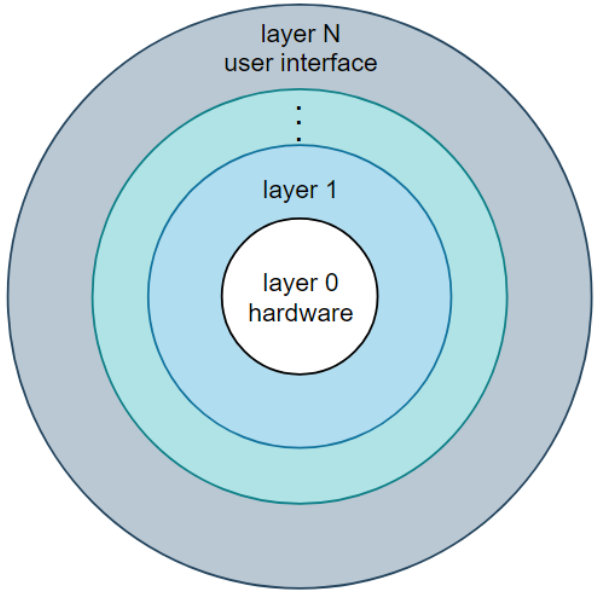
- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

Traditional UNIX System Structure



Layered Approach

- **The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.**
- **With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers**
- **Functions divided into Layers**
- **Each layer higher abstraction**
- **Hopefully don't skip layers**
  - Faster
  - Less portable



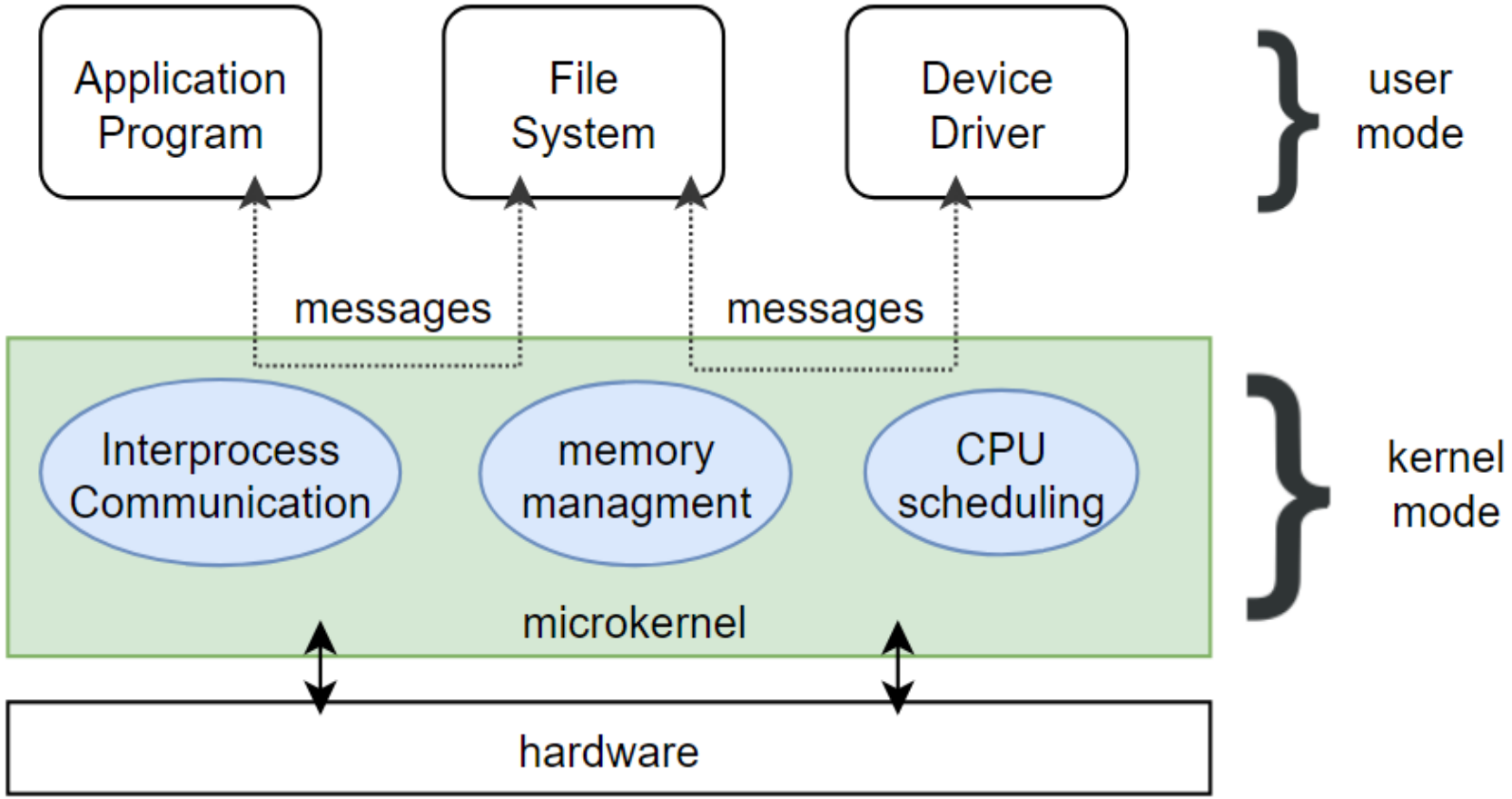
Monolithic Architecture

- **One dense module**
- **RAM was expensive**
- **Started adding features**
- **Got bloated**
  - Harder to add to
  - Harder to maintain

Microkernel Architecture

- **Moves as much from the kernel into user space**
- **Mach example of microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- **Communication takes place between user modules using message passing**
- **Benefits:**
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- **Detriments:**
  - Performance overhead of user space to kernel space communication

Microkernel System Structure



Minimalist vs. Maximalist

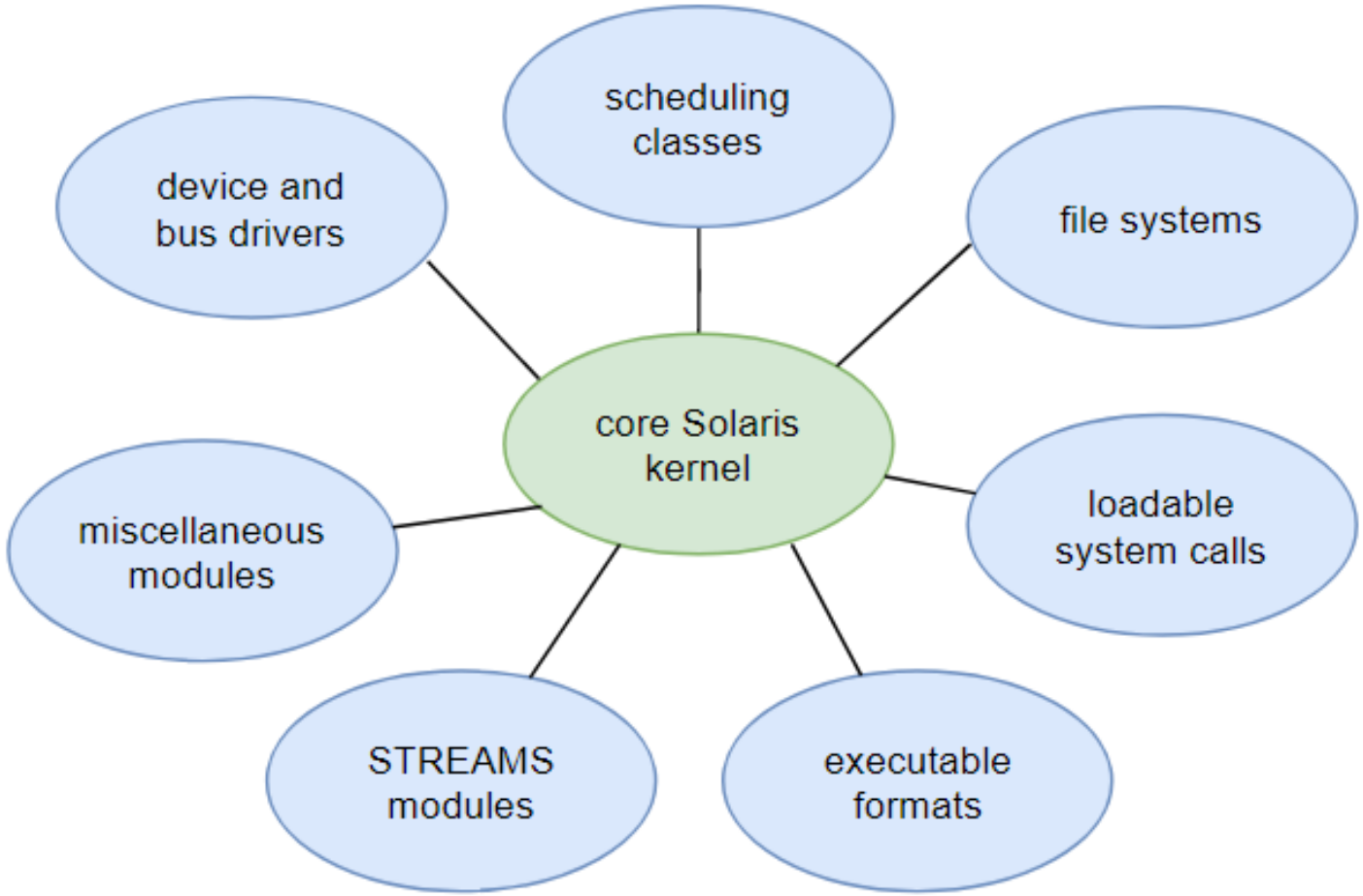
- **Less is more**
  - Minimum in kernel
  - Else in libraries and layers
  - More choices
- **All in One**
  - Operating Standards

Modules

- **Most modern operating systems implement loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel

- **Overall, similar to layers but with more flexible**
  - Linux, Solaris, etc

Solaris Modular Approach



Hybrid Systems

- **Most modern operating systems actually not one pure model**
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- **Apple macOS hybrid, layered, Aqua UI plus Cocoa programming environment**
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)

macOS Structure

Cocoa Application – Application User Interface responds to user events , manages app behavior.

App Kit									
Notification Center	Siri	Sharing	Full Screen Mode	Cocoa Autolayout	Popovers	Software Configuration	Accessibility	Apple Script	Spotlight

Media – Plays , records , editing audio-visual media , rendering 2D and 3D graphics.

AV Foundation	Core Animation	Core Audio	Core Image	Core Text	Open AL	Metal	Quartz
Audio playback, editing, analysis & recording.	2D rendering & animation. 3D Transformations.	Audio services for recording, playback and synchronization.	Fast image processing. Uses GPU based acceleration.	Handles Unicode fonts & texts.	Delivers 3D audio. High performance positional playbacks in games.	Portable 3D graphics apps & games. Imaging functions & effects.	macOS graphics, rendering support for 2D content. Event routing & cursor management.

Core Services – Fundamental Services for low level network communication , Automatic Reference Counting , Data Formatting , String Manipulation.

Address Book	Core Foundation	Quick Look	Security	Core Data	Foundation	Social	Webkit
Centralized database for contacts & groups.	Declares C based pro-grammatic interfaces. Data Types & Data Management.	Enables Spotlight & Finder to display thumbnail images.	User authentication, certificates & keys, authorization, keychain services etc.	Data model management & storage, undo/redo, validation of property values.	Swift framework for object behavior, inter-nationalization, data types & data mgmt.	Supports integration with social networking services.	Display HTML content in apps. Contains WebCore and JavaScript Core.

Core OS – Related to hardware and networking. Interfaces for running high-performance computation tasks on CPU or GPU.

Accelerate	Directory Services	Disk Arbitration	Metal	System Configuration
Accelerate complex operations, improve performance using vector unit, supports data parallelism, 3D graphic imaging, image processing.	Provides access to collected information about users, groups, computers, printers in a networked environment.	Notifies when local or remote volumes are mounted and unmounted.	Makes the high-performance parallel processing power of GPUs available to general purpose computing.	Provides access to current network configuration information. Determines reachability of remote hosts. Notifies about changes in network.

Kernel & Device Drivers – Device drivers & BSD Libraries , low level components. Support for file system security , interprocess communications.

BSD	File System	Mach	Networking
Provides basis for file systems and networking facilities, POSIX thread support, BSD sockets.	Supports multiple volume formats (NTFS, ExFAT, FAT, HFS+, APFS etc.) & file protocols (AFP, NFS etc.)	Protected Memory, Preemptive Multitasking, Advanced Virtual Memory, Real Time Support.	Supports network kernel extensions (NKEs). Create network modules, configure protocol stacks. Monitor and modify network traffic.

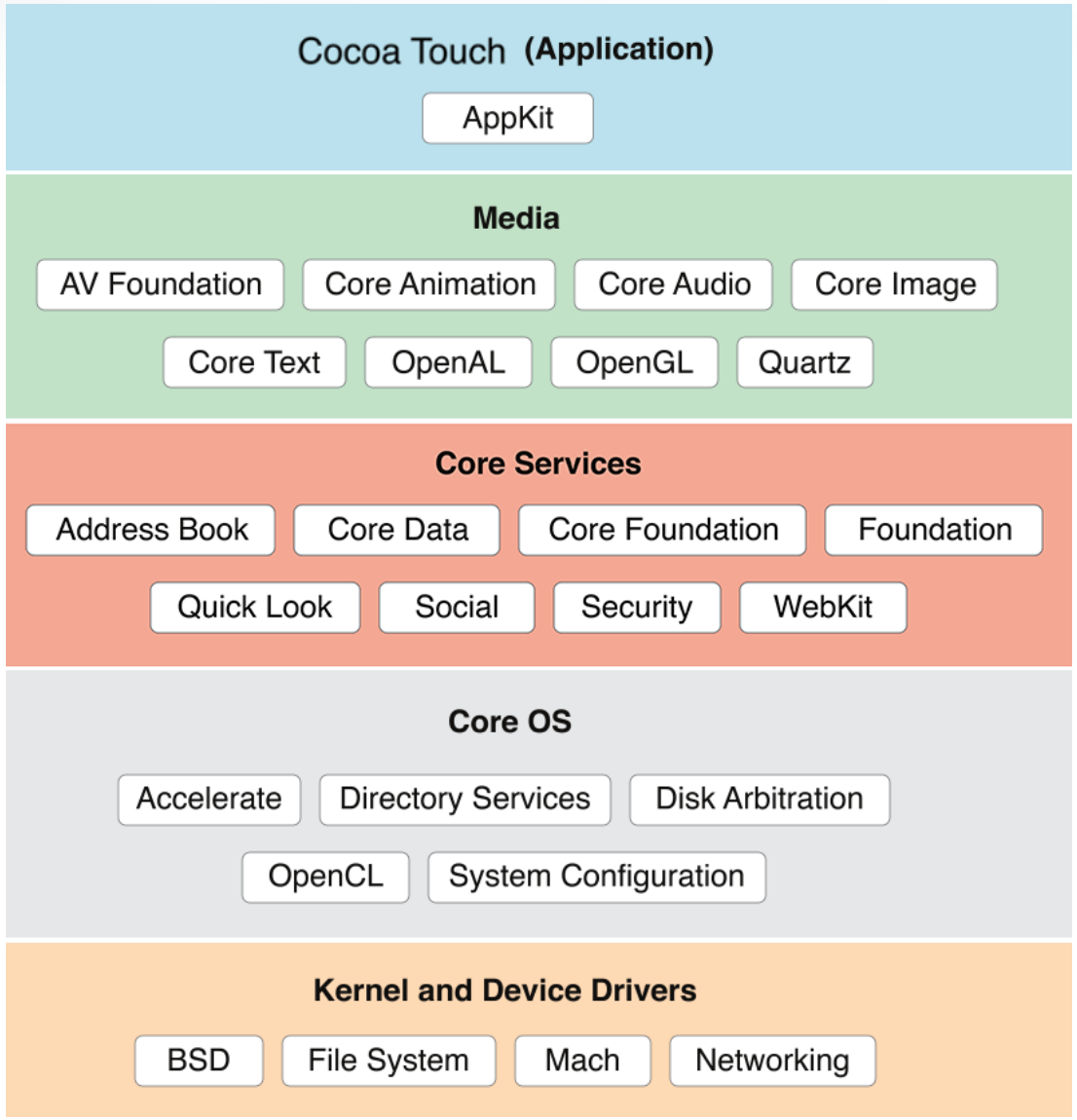
2017 update

iOS

Apple mobile OS for *iPhone*, *iPad*

- Structured on macOS, added functionality
- Does not run macOS applications natively
  - Also runs on different CPU architecture (ARM vs. Intel)
- **Cocoa Touch** Objective-C API for developing apps
- **Media** layer for graphics, audio, video
- **Core Services** consist of core services like Address book, Security, Social and foundation
- **Core OS** consist of technologies and frameworks which provide low-level services
- The lowest layer of iOS which mainly includes the kernel and device drivers.

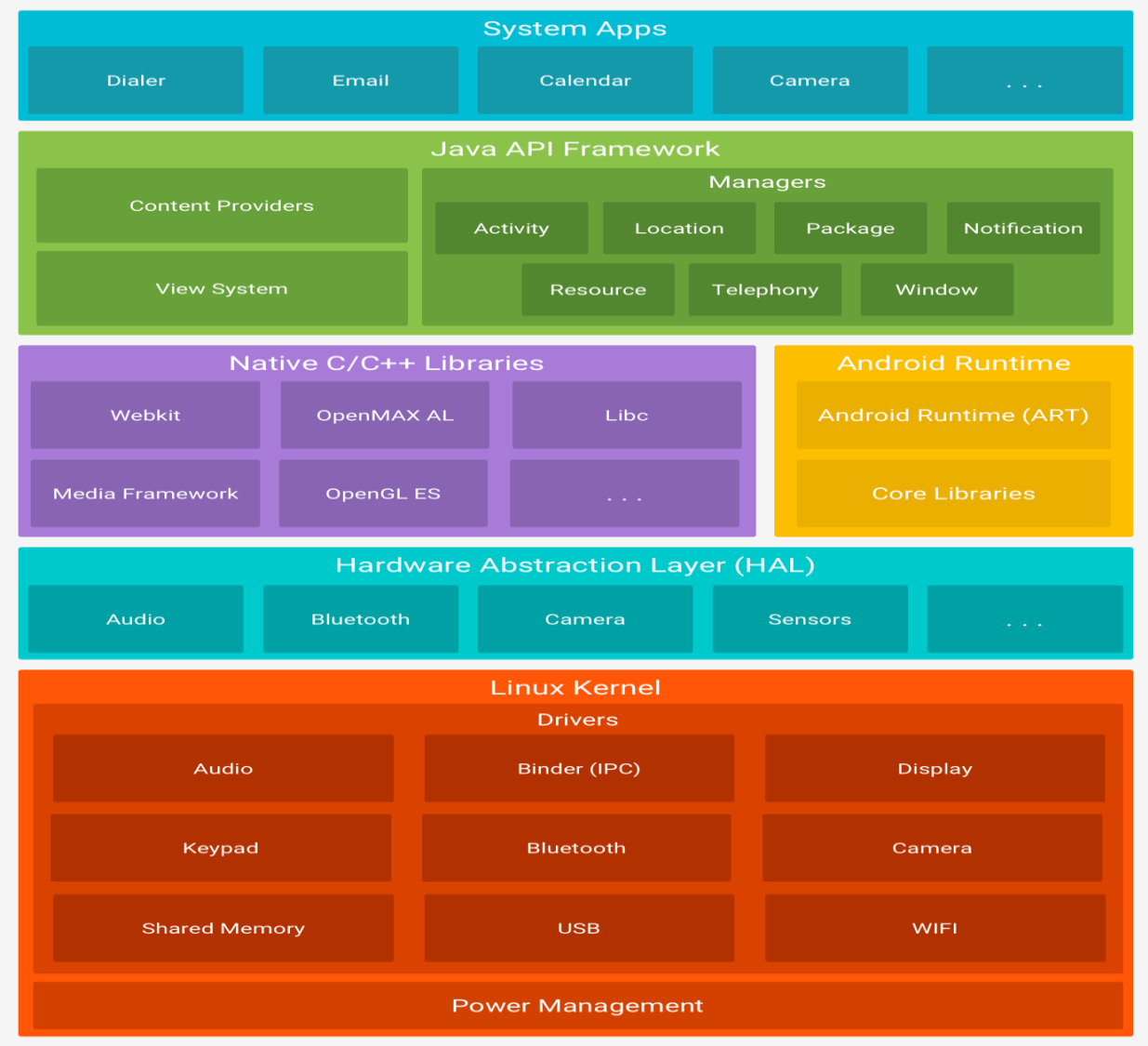
iOS


Cocoa Touch (Application)
AppKit
Media
AV Foundation
Core Animation
Core Audio
Core Image
Core Text
OpenAL
OpenGL
Quartz
Core Services
Address Book
Core Data
Core Foundation
Foundation
Quick Look
Social
Security
WebKit
Core OS
Accelerate
Directory Services
Disk Arbitration
OpenCL
System Configuration
Kernel and Device Drivers
BSD
File System
Mach
Networking

Android

- **Developed by Open Handset Alliance (mostly Google)**
  - Open Source
- **Similar stack to IOS**
- **Based on Linux kernel but modified**
  - Provides process, memory, device-driver management
  - Adds power management
- **Runtime environment includes core set of libraries and Dalvik virtual machine**
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM
- **Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc**

Android Architecture



Last updated:2020



Thanks for listening